`validPhase` specified in the Ballot smart contract uses a `require()` clause for checking the condition inside it and, if it fails, reverts the transaction. You also saw the use of `revert()` in the `vote()` function for validation that the voter has not already voted.

In the Ballot example, a single modifier was used in the function headers to validate the system parameters. Recall that the `validPhase` modifier enforces that all three functions--`vote()`, `register()`, and `reqWinner()`--are in the correct phase when they are invoked. Let's now define one more modifier to reinforce your understanding of modifiers. You'll do this for the validation within the `register()` function of the same smart contract. The modifier definition and use are shown in the next listing. Recall from the problem statement (section 3.2) that only the chairperson can register other voters. You can enforce this rule by using the `onlyChair` modifier.

Listing 3.6 Definition and use of `onlyChair` modifier

```
if (msg.sender != chairperson ..)                                          ❶


   modifier onlyChair ()                                                   ❷
   { require(msg.sender == chairperson);
  _;
}

function register(address voter) public validPhase(Phase.Regs) onlyChair   ❸
{
```

❶ Statement to be replaced by modifier onlyChair

❷ Modifier onlyChair definition

❸ Use of two modifiers in register() function's header

You apply multiple modifiers to a function by specifying them in a whitespace-separated list. Modifiers are evaluated in the order in which they're presented, so if the outcome of one modifier